

Guía Práctica: Transacciones en SQL

Control de Integridad y Concurrencia

April 23, 2026

1 Introducción

Una **transacción** es una unidad lógica de trabajo que comprende una o más sentencias SQL. Para que una base de datos sea considerada fiable, debe cumplir las propiedades **ACID**:

- **Atomicidad:** Todo o nada. Si una parte falla, falla toda la transacción.
- **Consistencia:** La base de datos pasa de un estado válido a otro estado válido.
- **Aislamiento (Isolation):** Las transacciones no se interfieren entre sí.
- **Durabilidad:** Una vez confirmado (COMMIT), el cambio es permanente.

2 Entorno de Pruebas

Para ejecutar los ejemplos de esta guía, trabajaremos con la base de datos **fp_transacciones**. Este esquema simula un entorno empresarial y educativo real con los siguientes componentes:

- **Gestión Académica (alumnos):** Registro de estudiantes con sus estados y notas.
- **Inventario (almacen):** Control de stock de productos tecnológicos.
- **Facturación (facturas, factura_detalle):** Relación maestro-detalle para simular ventas y pedidos.
- **Recursos Humanos (departamentos, empleados):** Estructura jerárquica con salarios y departamentos.
- **Seguridad y Auditoría (usuarios, historial_conexiones, auditoria_precios):** Tablas para registrar quién accede y qué cambios críticos se realizan.

Primero, prepara el entorno ejecutando el siguiente script:

```

-- =====
-- 00_setup.sql
-- Script de configuración del entorno de pruebas para Transacciones
-- =====

DROP DATABASE IF EXISTS fp_transacciones;
CREATE DATABASE fp_transacciones;
USE fp_transacciones;

-- -----
-- 1. Limpieza de tablas (Orden respetando restricciones de integridad)
-- -----

DROP TABLE IF EXISTS auditoria_precios;
DROP TABLE IF EXISTS historial_conexiones;
DROP TABLE IF EXISTS factura_detalle;
DROP TABLE IF EXISTS facturas;
DROP TABLE IF EXISTS empleados;
DROP TABLE IF EXISTS departamentos;
DROP TABLE IF EXISTS pedidos;
DROP TABLE IF EXISTS almacen;
DROP TABLE IF EXISTS alumnos;
DROP TABLE IF EXISTS modulos;
DROP TABLE IF EXISTS usuarios;

-- -----
-- 2. Creación de Tablas Maestras y Entidades
-- -----

CREATE TABLE alumnos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    estado VARCHAR(20) NOT NULL DEFAULT 'Activo',
    nota DECIMAL(4,2),
    CONSTRAINT chk_nota CHECK (nota BETWEEN 0 AND 10)
);

CREATE TABLE almacen (
    id INT PRIMARY KEY,
    producto VARCHAR(50) NOT NULL UNIQUE,
    stock INT NOT NULL DEFAULT 0,
    CONSTRAINT chk_stock_positivo CHECK (stock >= 0)
);

CREATE TABLE pedidos (
    id INT PRIMARY KEY,
    estado VARCHAR(20) NOT NULL DEFAULT 'Pendiente'
);

CREATE TABLE departamentos (
    id INT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL UNIQUE
);

```

```
CREATE TABLE usuarios (
    id INT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL UNIQUE
);
```

```
CREATE TABLE modulos (
    codigo VARCHAR(10) PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL
);
```

-- 3. Creación de Tablas con Claves Foráneas y Detalles

```
CREATE TABLE empleados (
    id INT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    salario DECIMAL(8,2) NOT NULL,
    departamento_id INT NOT NULL,
    CONSTRAINT fk_empleado_departamento
        FOREIGN KEY (departamento_id) REFERENCES departamentos(id),
    CONSTRAINT chk_salario_positivo
        CHECK (salario > 0)
);
```

```
CREATE TABLE facturas (
    id INT AUTO_INCREMENT PRIMARY KEY,
    fecha DATE NOT NULL,
    pagado BOOLEAN NOT NULL DEFAULT 0
);
```

```
CREATE TABLE factura_detalle (
    id INT AUTO_INCREMENT PRIMARY KEY,
    factura_id INT NOT NULL,
    producto VARCHAR(50) NOT NULL,
    CONSTRAINT fk_detalle_factura
        FOREIGN KEY (factura_id) REFERENCES facturas(id)
);
```

```
CREATE TABLE historial_conexiones (
    id INT AUTO_INCREMENT PRIMARY KEY,
    usuario_id INT NOT NULL,
    CONSTRAINT fk_historial_usuario
        FOREIGN KEY (usuario_id) REFERENCES usuarios(id)
);
```

```
CREATE TABLE auditoria_precios (
    id INT AUTO_INCREMENT PRIMARY KEY,
    producto_id INT NOT NULL,
    fecha DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    accion VARCHAR(100) NOT NULL,
    CONSTRAINT fk_auditoria_producto
```

```

FOREIGN KEY (producto_id) REFERENCES almacen(id)
);

-- -----
-- 4. Inserción de Datos Iniciales
-- -----

INSERT INTO alumnos (nombre, estado, nota)
VALUES
    ('Ana', 'Activo', 8.5),
    ('Luis', 'Inactivo', 4.0);

INSERT INTO almacen (id, producto, stock)
VALUES
    (1, 'Switch Cisco', 50),
    (2, 'Router Mikrotik', 30);

INSERT INTO pedidos (id, estado)
VALUES (1, 'Pendiente');

INSERT INTO departamentos (id, nombre)
VALUES
    (10, 'Contabilidad'),
    (20, 'Sistemas');

INSERT INTO empleados (id, nombre, salario, departamento_id)
VALUES
    (1, 'Carlos', 2000, 10),
    (2, 'Marta', 2200, 20);

INSERT INTO usuarios (id, nombre)
VALUES (5, 'admin');

INSERT INTO historial_conexiones (usuario_id)
VALUES (5), (5);

```

3 Nivel 1: Sintaxis Básica y DML Simple

En MySQL/MariaDB, por defecto, cada sentencia se ejecuta en modo *autocommit*. Para agrupar varias sentencias en una transacción, debemos usar **START TRANSACTION**.

3.1 01. Alta segura de un registro

El uso de **COMMIT** hace que los cambios sean persistentes.

```

-- 01. Alta segura de un registro
START TRANSACTION;
INSERT INTO alumnos (nombre, estado) VALUES ('Elena', 'Activo');
SELECT * FROM alumnos WHERE nombre = 'Elena';
COMMIT;

```

Reflexión 1

¿Por qué es fundamental ejecutar un `SELECT` antes del `COMMIT` en una transacción manual?

3.2 02. El botón de pánico: `ROLLBACK`

Si detectamos un error antes de confirmar, `ROLLBACK` deshace todos los cambios desde el inicio de la transacción.

```
-- 02. Prueba y descarte (El botón de pánico)
START TRANSACTION;
INSERT INTO alumnos (nombre, estado) VALUES (NULL, 'Error Intencionado');
ROLLBACK;
-- Comprueba que el registro nulo no existe:
SELECT * FROM alumnos;
```

Reflexión 2

En el ejemplo 02, si no ejecutáramos `ROLLBACK` ni `COMMIT` y cerráramos la terminal, ¿qué ocurriría con el registro del alumno?

3.3 03. Actualización controlada

Las transacciones permiten verificar los cambios antes de hacerlos públicos.

```
-- 03. Actualización controlada
START TRANSACTION;
UPDATE pedidos SET estado = 'Enviado' WHERE id = 1;
SELECT * FROM pedidos WHERE id = 1;
COMMIT;
```

Reflexión 3

¿Qué sucede si el `WHERE` de un `UPDATE` no coincide con ninguna fila? ¿Se considera un error que aborta la transacción?

3.4 04. Prevención de desastres

Las transacciones son nuestra red de seguridad al ejecutar `UPDATE` o `DELETE` masivos.

```
-- 04. Prevención de desastres en UPDATE
START TRANSACTION;
UPDATE empleados SET salario = 0; -- ¡Olvidamos el WHERE!
SELECT * FROM empleados;
ROLLBACK; -- Salvamos los datos
```

Reflexión 4

Si olvidas el `WHERE` en un `UPDATE`, ¿cómo ayuda la transacción a recuperar la información?

3.5 05. Simulación de borrado (Soft-delete)

El borrado lógico es una alternativa segura al borrado físico de registros.

```
-- 05. Simulación de borrado (Soft-delete manual)
START TRANSACTION;
DELETE FROM alumnos WHERE estado = 'Inactivo';
SELECT * FROM alumnos; -- Aquí ya no sale Luis
ROLLBACK;
```

Reflexión 5

¿Por qué es preferible el "borrado lógico" (*soft-delete*) en sistemas que requieren trazabilidad e histórico?

4 Nivel 2: Lógica Transaccional y Relaciones

4.1 06. Transferencia de valores

Clásico ejemplo de **atomicidad**: si el primer UPDATE funciona pero el segundo falla, el ROLLBACK asegura que no se pierda stock por el camino.

```
-- 06. Transferencia de valores (Stock entre almacenes)
START TRANSACTION;
UPDATE almacen SET stock = stock - 10 WHERE id = 1;
UPDATE almacen SET stock = stock + 10 WHERE id = 2;
COMMIT;
```

Reflexión 6

Si el segundo UPDATE falla por una restricción CHECK (ej. stock negativo), ¿se guarda el primer UPDATE automáticamente?

4.2 07. Alta Maestro-Detalle

Garantiza que no existan facturas sin sus correspondientes líneas de detalle.

```
-- 07. Alta Maestro-Detalle (Uso de LAST_INSERT_ID)
START TRANSACTION;
INSERT INTO facturas (fecha) VALUES (CURDATE());
SET @factura_id = LAST_INSERT_ID();
INSERT INTO factura_detalle (factura_id, producto) VALUES (@factura_id, 'Monitor
24"), (@factura_id, 'Teclado Mecánico');
COMMIT;
```

Reflexión 7

¿Qué riesgo correríamos si insertáramos la factura y sus detalles en sentencias separadas sin una transacción?

4.3 08. Baja en Cascada Manual

Cuando no tenemos configurado el borrado automático, debemos gestionar nosotros la limpieza de tablas hijas.

```
-- 08. Baja en Cascada Manual (Sin ON DELETE CASCADE configurado)
START TRANSACTION;
DELETE FROM historial_conexiones WHERE usuario_id = 5;
DELETE FROM usuarios WHERE id = 5;
COMMIT;
```

Reflexión 8

¿Por qué es vital usar una transacción al realizar un borrado manual en cascada de varias tablas relacionadas?

4.4 09. Auditoría y Logs

Asegura que cada cambio importante en los datos quede registrado en una tabla de auditoría en la misma operación atómica.

```
-- 09. Actualización con registro de auditoría (Log)
START TRANSACTION;
UPDATE almacen SET stock = stock + 50 WHERE id = 1;
INSERT INTO auditoria_precios (producto_id, fecha, accion) VALUES (1, NOW(), '
  Reposición de 50 unidades de Switch');
COMMIT;
```

Reflexión 9

¿Qué ocurriría si el registro de auditoría fallara pero el cambio principal tuviera éxito en una transacción?

5 Conceptos Críticos y Peligros

5.1 10. El Peligro del DDL (Commit Implícito)

¡Atención!

Las sentencias DDL (CREATE, DROP, ALTER, TRUNCATE) provocan un **COMMIT implícito**. No se pueden deshacer con un ROLLBACK.

```
-- 10. Cierre implícito traicionero (El peligro del DDL)
-- Las sentencias DDL provocan un COMMIT automático.
START TRANSACTION;
UPDATE almacen SET stock = stock - 1 WHERE id = 1;
-- Alguien decide crear una tabla temporal o truncar otra en medio:
TRUNCATE TABLE modulos;
ROLLBACK;
-- Resultado: El ROLLBACK no tiene efecto sobre el UPDATE.
-- El TRUNCATE forzó un COMMIT implícito previo.
```

Reflexión 10

¿Por qué el motor de base de datos fuerza un COMMIT al ejecutar un TRUNCATE o un ALTER TABLE?

5.2 11. Falsas Transacciones Anidadas

MySQL no soporta transacciones anidadas. Ejecutar un segundo START TRANSACTION cierra automáticamente el anterior.

```
-- 11. Transacciones "Anidadas" (El riesgo de un nuevo START TRANSACTION)
START TRANSACTION;
UPDATE alumnos SET nota = 10.00 WHERE id = 1;
-- Un script mal diseñado intenta abrir otra transacción interna:
START TRANSACTION; -- ¡CUIDADO! Esto provoca un COMMIT implícito de la anterior.
UPDATE alumnos SET estado = 'Matrícula de Honor' WHERE id = 1;
ROLLBACK;
-- Solo el segundo cambio se deshace. El 10.00 se guardó.
```

Reflexión 11

¿Qué técnica se debe usar en MySQL si realmente necesitamos puntos de retorno parciales dentro de una misma transacción?

6 Niveles de Aislamiento y Concurrency

En un entorno de producción real, el servidor de base de datos no es un sistema aislado. Cientos o miles de usuarios acceden simultáneamente.

El Reto de la Concurrency

Cuando dos o más transacciones intentan modificar o leer los mismos datos al mismo tiempo, surgen conflictos. El **Aislamiento** gestiona estas colisiones.

6.1 El Balance: Velocidad vs. Seguridad

No existe el "nivel perfecto". La gestión implica elegir un punto en una balanza:

- **Máxima Seguridad (SERIALIZABLE):** Evita cualquier anomalía, pero es lento por los bloqueos.
- **Máxima Velocidad (READ UNCOMMITTED):** Extremadamente rápido, pero permite "Lecturas Sucias".

6.2 Los 4 Niveles de Aislamiento

1. **READ UNCOMMITTED:** Permite leer cambios no confirmados (*Lecturas Sucias*).
2. **READ COMMITTED:** Solo lee datos confirmados. Evita lecturas sucias, pero permite *Lecturas No Repetibles*.

3. **REPEATABLE READ:** Garantiza que los datos no cambien durante tu transacción. Usa *Snapshots*.
4. **SERIALIZABLE:** El aislamiento total. Bloquea rangos de filas para evitar *Lecturas Fantasma*.

Duelo de Motores: Valores por Defecto

MySQL (InnoDB): REPEATABLE READ (Prioriza consistencia de lectura mediante MVCC).

Oracle y PostgreSQL: READ COMMITTED (Priorizan concurrencia y rendimiento).

6.3 12. Lectura Sucia (Dirty Read)

Ocurre en el nivel **READ UNCOMMITTED**. Una transacción lee datos que otra aún no ha confirmado.

```
-- 12. Demostración de Lectura Sucia (Dirty Read)
SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
-- Terminal 1:
START TRANSACTION;
UPDATE alumnos SET nota = 0.00 WHERE id = 1;
-- Terminal 2 (Paralela):
SELECT nota FROM alumnos WHERE id = 1;
-- Terminal 1:
ROLLBACK;
```

Reflexión 12

¿Cuál es el riesgo real de que un sistema contable permita lecturas sucias durante el cierre del mes?

6.4 13. Lectura Repetible (Repeatable Read)

Es el nivel por defecto en InnoDB. Garantiza consistencia de lectura durante toda la transacción.

```
-- 13. Repeatable Read (Nivel por defecto en InnoDB)
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
-- Terminal 1:
START TRANSACTION;
SELECT nota FROM alumnos WHERE id = 1;
-- Terminal 2:
START TRANSACTION;
UPDATE alumnos SET nota = 5.00 WHERE id = 1;
COMMIT;
-- Terminal 1:
SELECT nota FROM alumnos WHERE id = 1;
COMMIT;
```

Reflexión 13

Si REPEATABLE READ es más seguro para la consistencia, ¿qué ganamos al bajar el aislamiento a READ COMMITTED?

6.5 14. Deadlock (Abrazo Mortal)

Situación donde dos procesos se bloquean mutuamente esperando recursos que el otro tiene.

```
-- 14. El temido Abrazo Mortal (Deadlock)
-- Terminal 1:
START TRANSACTION;
UPDATE almacen SET stock = 100 WHERE id = 1;
-- Terminal 2:
START TRANSACTION;
UPDATE almacen SET stock = 200 WHERE id = 2;
-- Terminal 1 (Espera a B):
UPDATE almacen SET stock = 300 WHERE id = 2;
-- Terminal 2 (Provoca Deadlock):
UPDATE almacen SET stock = 400 WHERE id = 1;
```

6.6 15. SERIALIZABLE

El nivel más estricto. Convierte todas las lecturas en bloqueos para evitar incluso las "lecturas fantasma".

```
-- 15. Aislamiento SERIALIZABLE (Evitando Lecturas Fantasma)
SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
-- Terminal 1:
START TRANSACTION;
SELECT * FROM alumnos WHERE nota > 8;
-- Terminal 2:
START TRANSACTION;
INSERT INTO alumnos (nombre, nota) VALUES ('Nuevo Genio', 9.5);
-- Resultado: Terminal 2 se queda bloqueada esperando que T1 libere el rango.
```

7 Respuestas a las Reflexiones e Investigación

Respuesta 1: Verificación Humana

Para verificar que los cambios que vamos a confirmar son exactamente los deseados. En una transacción manual, el `SELECT` nos permite actuar como "auditores" antes de dar el paso irreversible del `COMMIT`.

Respuesta 2: El Rollback Implícito

Se perdería. Si la conexión se corta o el cliente se cierra sin un `COMMIT` explícito, el servidor realiza un `ROLLBACK` automático por seguridad para mantener la integridad.

Respuesta 3: El estado de la ejecución

No se considera un error. SQL ejecutará la sentencia, informará de que "0 filas fueron afectadas" y la transacción continuará con éxito. La transacción solo abortaría ante errores de sintaxis, violaciones de integridad o desconexiones.

Respuesta 4: Red de seguridad

Permite revisar el desastre localmente y usar `ROLLBACK` para volver al estado inicial como si nada hubiera pasado. Sin transacción, el cambio sería inmediato y definitivo.

Respuesta 5: Trazabilidad

Porque permite mantener el histórico para auditorías, permite "deshacer" el borrado simplemente cambiando el estado y evita la rotura de integridad referencial en tablas que guardan históricos antiguos.

Respuesta 6: Atomicidad

No. Aunque la sentencia del primer `UPDATE` haya sido correcta, el fallo de cualquier sentencia posterior dentro de la misma transacción (o el `ROLLBACK` manual) invalida todo el bloque. **Todo o nada.**

Respuesta 7: Datos Huérfanos

Podríamos acabar con una factura en la tabla `facturas` pero, si el sistema falla antes de insertar los detalles, tendríamos una factura vacía, rompiendo la lógica de negocio.

Respuesta 8: Consistencia Relacional

Si borramos el padre pero falla el borrado de los hijos (o viceversa), dejaríamos la base de datos en un estado inconsistente con registros "huérfanos" o referencias a registros que ya no existen.

Respuesta 9: Integridad del Log

Al estar en la misma transacción, si el insert en la auditoría falla, todo el proceso (incluyendo el cambio principal) se deshacería. Así garantizamos que no haya cambios sin su correspondiente rastro en el log.

Respuesta 10: Cambios Estructurales

Porque las sentencias DDL modifican el diccionario de datos (la estructura física de los archivos en disco). Gestionar un rollback de un cambio de estructura mientras otros usuarios acceden es extremadamente complejo y costoso.

Respuesta 11: Savepoints

Se deben utilizar los `SAVEPOINT nombre_punto`, que permiten hacer `ROLLBACK TO SAVEPOINT` sin abortar la transacción completa.

Respuesta 12: Información Volátil

Se podrían generar informes financieros basados en datos que fueron cancelados o corregidos segundos después, provocando descuadres legales o bancarios graves.

Respuesta 13: Rendimiento vs Concurrency

Ganamos **concurrency**. Los niveles altos requieren mantener más versiones de los datos en memoria o poner más bloqueos, lo que puede ralentizar el sistema ante miles de usuarios simultáneos.